ENSIIE

Projet de Programmation Impérative

Implémentation d'une Machine Virtuelle pour Bytecode OCaml

Auteur :
EL KHAZROUNI
Hicham

Enseignant :
M. Guillaume Burel

26 mars 2026

Table des matières

1 Introduction **2**

2 Architecture de la Machine Virtuelle **2**
2.1 Structure de données principale 2
2.2 Représentation et encodage des données 3
2.3 Fonctionnement général du programme 3

3 Implémentation des Instructions **4**
3.1 Mes choix 4
3.2 Le cycle d'exécution 4
3.3 Problèmes rencontrés 5
3.4 Solutions trouvées 6

4 Tests et Validation **7**

5 Conclusion et Optimisations **7**

1 Introduction

L'objectif de ce projet est la réalisation d'une machine virtuelle capable d'interpréter et d'exécuter un jeu d'instructions (bytecode) spécifique. Ce système repose sur une architecture à pile complétée par un accumulateur, simulant ainsi le comportement de bas niveau d'un langage fonctionnel comme OCaml.

2 Architecture de la Machine Virtuelle

2.1 Structure de données principale

La machine virtuelle est définie par une structure regroupant les éléments essentiels :

- **Tableau de codes** : Un tableau d'entiers représentant les instructions et les valeurs en argument de celles-ci.
- **c** : La taille du tableau de codes
- **Tableau de valeurs gloables** : Un tableau de valeurs globales accessible partout pour stocker des informations.
- **v** : La taille du tableau de valeurs globales
- **Accumulateur** : Une variable contenant des valeurs.
- **Pile** : Utilisée pour le stockage temporaire et le passage d'arguments.
- **Indice** : Le compteur qui permet de savoir l'avancement du programme dans le code
- **running** : une variable qui permet de connaître l'état de marche de la machine.

L'implémentation de ceci a été réalisée comme ci-dessous :

```
struct virtual_machine {
    unsigned int c; // nombre de codes dans le tableau de codes
    unsigned int v; // nombre de valeurs dans le tableau de valeurs globales

    int* codes; // tableau des codes
    long* valeurs; // tableau des valeurs globales

    unsigned int indice; // indice courant dans le tableau de codes
    long accu; // accumulateur
    int running; // 1 ou 0 selon l'etat de la machine virtuelle
```

```
    pile p; // pile de valeurs
};
```

2.2 Représentation et encodage des données

Une particularité du projet réside dans l'encodage des entiers. Pour distinguer les entiers des pointeurs :

$$\text{valeur_encodée} = 2 \times n + 1 \tag{1}$$

Les valeurs contenues dans la pile, dans l'accumulateur, dans le tableau de valeurs globales sont encodées mais pas celles dans le tableau de codes. Cela permet de différencier les pointeurs des valeurs.

2.3 Fonctionnement général du programme

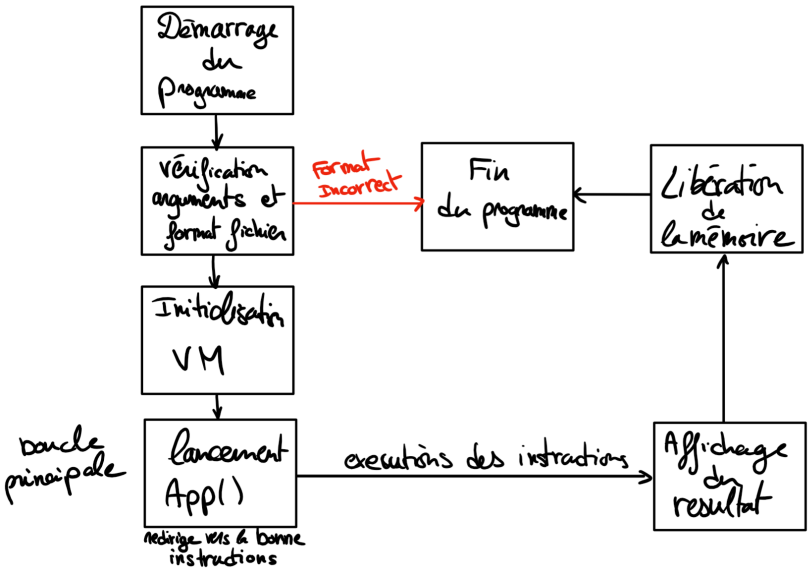

FIGURE 1 – Schéma explicatif du fonctionnement du programme

App() sera developpé dans une partie plus tard. Il faut juste savoir qu'elle gére le lancement des instructions.

3 Implémentation des Instructions

3.1 Mes choix

Toutes les données concernant la machine virtuelle sont regroupées dans une structure. Cela permet une importation des données et une modification simplifiée par les instructions, ne forçant pas à retourner des tableaux ou refaire des affectations.

La vérification du format du fichier d'entrée n'étant effectuée qu'une seule fois et étant utilisée uniquement dans le main.c j'ai pris l'initiative de faire cette vérification directement dans le main.

La déclaration des structures a été faite plusieurs fois dans différents fichiers car je ne voulais pas les déclarer dans un .h car on est censé y mettre uniquement les prototypes de fonction et déclarer l'existence d'un type.

Certaines instructions peuvent faire planter la machine car par manque d'information, certaines gestions d'erreurs sont impossible, j'ai donc décidé de prévenir uniquement dans ce rapport. J'en parlerai plus en détail dans la partie des problèmes rencontrés.

Certaines instructions étant répétitives, je me suis permis de faire des fonctions plus abstraites à paramètre comme ACCN pour ne pas répéter le même code pour ACC1, ACC2, ACC3, etc.

La gestion d'indice suit une règle simple pour ne pas produire d'erreur lors d'ajout d'instructions supplémentaire. La règle d'or est de faire l'incrémentation d'indice seulement et uniquement à la fin de l'instruction. Lors d'appel d'instructions dans une autre, il faut donc compenser cette incrémentation juste après l'appel pour ensuite l'incrémenter à la fin. Cela permet de converver un indice fixe durant toute l'exécution d'une instruction et facilite la compréhension en relecture.

Par ailleurs j'ai considéré qu'un fichier dont la deuxième ligne donne une taille de tableau de codes nulle comme une erreur de format.

3.2 Le cycle d'exécution

Le cœur du programme se situe dans une boucle `while` qui appelle la fonction `App()`. Cette dernière utilise une structure `switch/case` pour rou-

ter chaque code vers son instruction correspondante. Les instructions modifie directement l'état de la machine virtuelle, il suffit donc de vérifier l'état de fonctionnement de la machine et l'indice pour savoir si on arrête de boucler dans le cas d'erreur, de terminaison du tableau de code ou d'appel de l'instruction STOP.

3.3 Problèmes rencontrés

La lecture du fichier m'a posé plusieurs problèmes déjà pour la gestion d'erreur mais aussi pour récupérer les valeurs contenues dedans. En effet dans un premier temps j'utilisais un fprintf(stderr, ...) suivi d'un return dans mon main.c pour annoncer un problème d'accès au fichier mais cela ne terminait pas le programme et m'affichait en plus de mon message d'erreur un "Segmentation Fault". Ma première version pour récupérer les valeurs des tailles des tableaux utilisait la taille maximale d'un type int comme raison de terminaison de boucle et avait des fuites mémoires car je concaténais des str à l'aide de malloc() succéssifs sans free().

La compréhension du sujet m'a été particulièrement difficile car les informations importantes étaient fragmentées au début et à la fin du sujet mais aussi car certaines parties n'était pas très claires, comme l'encodage des valeurs où j'ai eu beaucoup de mal à comprendre ce qui était encodé. Ce qui m'a valu d'énormes problèmes lors du debogage. En effet je pensais que les valeurs présentes comme arguments d'instructions dans le tableau de codes, étant dans valeurs, étaient encodées elles aussi. Mais non, ce qui me faisait faire des sauts d'indice incohérents, récupération de valeurs incohérentes, etc.

La gestion des primitives avec des types d'entrées différents et des retours différents m'a été difficile aussi car je pensais qu'il suffisait de faire une fonction qui gérait tout et que ça allait bien se faire mais comme on doit annoncer le type de retour de nos fonctions et qu'il doit être unique, j'ai dû penser et schématiser des appels de primitives pour trouver comment faire.

Ma mauvaise compréhension du sujet m'a aussi fait commencer l'implémentation des instructions en pensant que l'accumulateur était du type int, alors qu'il doit pouvoir accepter des pointeurs, j'ai dû donc revoir l'intégralités de ce que j'avais fait lorsque je me suis rendu compte de cela. Une erreur de logique dans ma fonction de dépilement de pile m'a forcé à retoucher tout le code aussi. Mon erreur a été de ne pas penser que ma valeur de retour qui indique l'erreur : -1 pouvait être retournée aussi par la pile.

On ne peut connaître la taille d'un tableau quelconque si on ne la connaît pas d'avance, ce qui fait que pour les instructions d'écriture et accès aux blocs mémoires on peut avoir des problèmes. Il est impossible de savoir si on prend une valeur qui est bien dans les bornes du tableau pour pouvoir faire une gestion d'erreur.

Par ailleurs, j'ai dû travailler sur différents ordinateurs lors du projet et pour la synchronisation, je n'ai pas utilisé d'outils adaptés ce qui fait que j'ai perdu quelques fois des parties de mon implémentation.

3.4 Solutions trouvées

Pour la gestion d'erreur de la lecture du fichier, l'utilisation de perror() m'a permis de résoudre mon problème de Segfault. Pour la récupération des valeurs de tailles de tableaux j'ai pris une tout autre approche, qui consiste à une lecture caractère par caractère et de vérifier si ces caractère sont des entiers ou non. Sachant que je connais l'autre d'apparition des caractères non entiers autorisés : ' ' et '`\n`', j'ai pu faire en sorte de stocker les nombres qui étaient lus petit à petit. Cela a réglé mon problème de fuite mémoire en plus et simplifié mon code en retirant des boucles while.

Une grosse partie de relecture du sujet et de questionnement m'a permis de mieux comprendre le sujet même s'il restait des zones d'ombre jusqu'à tard.

L'appel à un gestionnaire de fonctions permettant de traiter et convertir les arguments passés ainsi que de gérer les valeurs retournées par les primitives a tout réglé.

L'erreur du dépilement de la pile a pu être résolu en passant en argument un pointeur vers une variable qui m'indique s'il y a eu une erreur lors de l'exécution de la fonction, cela me permet d'être sûr que l'indicateur d'erreur n'est pas mélangé avec les retours possibles de ma fonction.

Le problème de non connaissance de la taille du tableau n'est pas résolvable, il s'agit donc à l'utilisateur de bien gérer le bytecode et de ne pas donner de valeur qui fasse sortir du tableau.

L'utilisation de github m'aurait bien servi mais lorsque j'y ai pensé, j'allais utilisé uniquement un même ordinateur donc je le ferai pour le prochain projet. Pour ce projet j'ai utilisé un dossier avec des sauvegardes de version du projet rangé dans différents dossiers pour me permettre de revenir

à d'ancienne version en cas de suppression de donnée par un mauvaise synchronisation.

4 Tests et Validation

Le bon fonctionnement de la machine virtuelle a été validé par :
- `base.sobf` : Validation du fonctionnement de la pile
- `affichage.sobf` : Validation de lecture et écriture simple
- `branchs.sobf` : Validation des sauts et de la logique conditionnelle.
- `fact.sobf` : Validation d'écriture et des instructions de calculs
- `wumpus.sobf` : Affichage et lecture de données solides
- `pinetree.sobf` : Calculs et affichage solides
- `blocks.sobf` : tout fonctionne
- `ints.sobf` : pareil

5 Conclusion et Optimisations

Ce projet m'a permis de comprendre le lien entre le code source de haut niveau et son exécution matérielle par le programme de lecture.
De plus avoir de bons outils pour travailler sur différents ordinateurs lors d'un projet est crucial.

Optimisations possibles :
- Faire des fichiers tests pour chaque instruction pour vérifier de son bon fonctionnement même dans les cas limites.
- ajouter l'option de débogage pour notre machine virtuelle.
- ajouter d'autres instructions
- si demandé, ajouter la possibilité d'avoir un tableau de codes vide (je ne vois pas vraiment une grande utilité mais possible de le faire)